# Evidencias de funcionamiento
# Entrega 2P2
# Kafka e integración orientada a eventos

**Estudiantes:**
**Juan David Collazos Garzon**
**Didier Fabian Benavides Rojas**
**Paula Katherin Santamaria Fernandez**

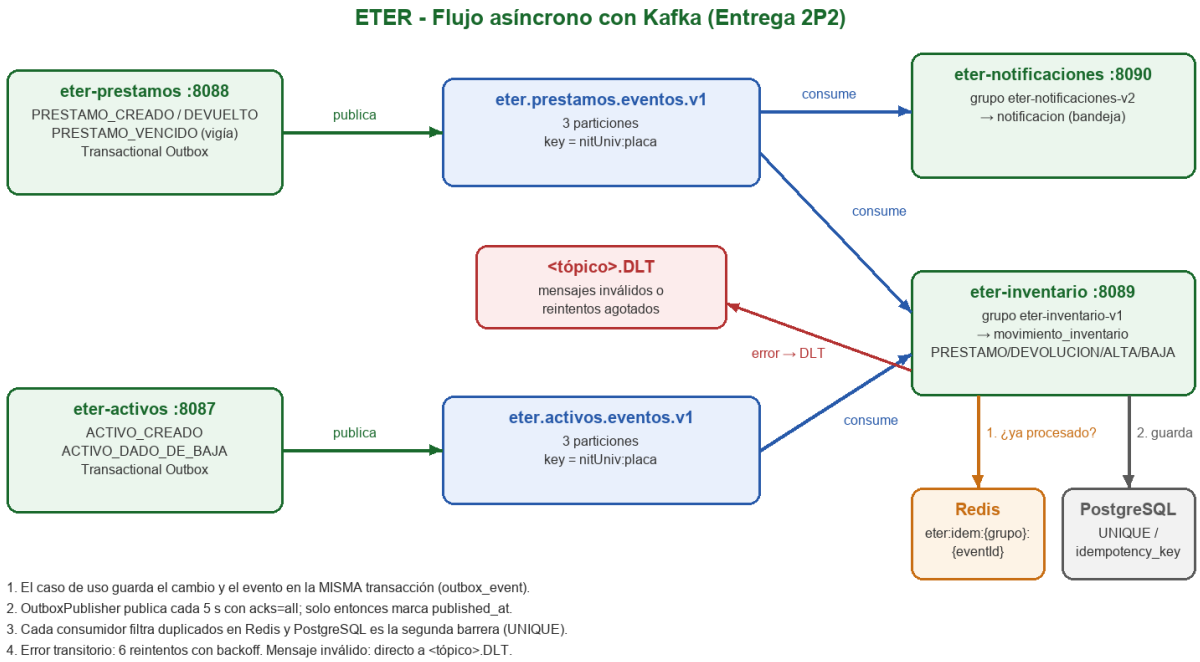
**Docente: Diego Fernando Vargas Ramirez**

**Fundación Escuela Tecnológica de Neiva**
**Programación Avanzada**
**03/10/2026**

## 1. Resumen de la entrega

En esta entrega se integró **Apache Kafka** al ecosistema ETER para resolver una necesidad real de la arquitectura aprobada: cuando ocurre un hecho de negocio sobre un activo (se registra, se presta, se vence el préstamo, se devuelve o se da de baja) **dos servicios distintos** deben reaccionar. Notificaciones avisa al custodio e Inventario registra el movimiento en el kardex. Antes, el kardex se llenaba a mano y podía quedar desfasado; ahora los productores publican el evento una vez y cada consumidor lo procesa de forma asíncrona, idempotente y tolerante a fallos.

| Componente | Rol en Kafka | Responsabilidad |
|---|---|---|
| **Activos :8087** | Productor | CRUD de activos. Publica ACTIVO_CREADO y ACTIVO_DADO_DE_BAJA (Outbox). |
| **Préstamos :8088** | Productor | Préstamos, devoluciones y vigía. Publica PRESTAMO_CREADO, PRESTAMO_VENCIDO y PRESTAMO_DEVUELTO (Outbox). |
| **Inventario :8089** | Consumidor | Grupo eter-inventario-v1. Lee ambos tópicos y registra PRESTAMO, DEVOLUCION, ALTA y BAJA en el kardex. |
| **Notificaciones :8090** | Consumidor | Grupo eter-notificaciones-v2. Lee préstamos y crea la notificación en la bandeja del custodio. |
| **Redis 7.4** | Idempotencia | Guarda eter:idem:{grupo}:{eventId} = PROCESADO (TTL 7 días). |
| **PostgreSQL 16 / Kafka 3.9** | Infraestructura | Un schema por servicio. Kafka en modo KRaft, tópicos creados por kafka-init. |




1. El caso de uso guarda el cambio y el evento en la MISMA transacción (outbox_event).
2. OutboxPublisher publica cada 5 s con acks=all; solo entonces marca published_at.
3. Cada consumidor filtra duplicados en Redis y PostgreSQL es la segunda barrera (UNIQUE).
4. Error transitorio: 6 reintentos con backoff. Mensaje inválido: directo a <tópico>.DLT.

*Figura 1. Arquitectura del flujo asíncrono implementado*

## 2. Diseño de tópicos y eventos

Se definió **un tópico por agregado** con sufijo de versión y un tópico DLT para cada uno. La key del mensaje es la identidad del activo (**nitUniv:placaInventario**), así todos los eventos de un mismo activo caen en la misma partición y se consumen en orden (no puede procesarse una devolución antes que el préstamo). La creación automática de tópicos está deshabilitada: solo existen los del diseño.

| Tópico | Part. | Productor | Consumidores (grupo) |
|---|---|---|---|
| `eter.prestamos.eventos.v1` | 3 | prestamos-service (Outbox) | eter-notificaciones-v2, eter-inventario-v1 |
| `eter.activos.eventos.v1` | 3 | activos-service (Outbox) | eter-inventario-v1 |
| `eter.prestamos.eventos.v1.DLT` | 3 | Error handler de cada consumidor | Revisión manual (Kafka UI) |
| `eter.activos.eventos.v1.DLT` | 3 | Error handler de cada consumidor | Revisión manual (Kafka UI) |

| Evento | Cuándo se emite | Efecto en los consumidores |
|---|---|---|
| **PRESTAMO_CREADO** | POST /prestamos exitoso | Inventario: movimiento PRESTAMO. Notificaciones: “Préstamo registrado”. |
| **PRESTAMO_VENCIDO** | El vigía detecta la fecha pactada vencida (una sola vez) | Notificaciones: “Préstamo vencido”. Inventario lo ignora. |
| **PRESTAMO_DEVUELTO** | PATCH /prestamos/{id}/devolucion | Inventario: DEVOLUCION. Notificaciones: “Préstamo devuelto”. |
| **ACTIVO_CREADO** | POST /activos-fijos exitoso | Inventario: movimiento ALTA. |
| **ACTIVO_DADO_DE_BAJA** | DELETE /activos-fijos (si no estaba en BAJA) | Inventario: movimiento BAJA. |

### Estructura del evento

Todos los mensajes comparten un sobre con **eventId** (clave de idempotencia), **eventType**, **eventVersion**, **source**, **assetKey**, **nitUniv**, **placaInventario**, **actor**, **occurredAt** y el **payload** propio del tipo. El documento completo está en docs/EVENTOS_KAFKA.md. Este es un mensaje real leído del tópico:

Mensaje real en eter.prestamos.eventos.v1

```
// topico=eter.prestamos.eventos.v1  Partition:1  Offset:9  key=900123456-1:AF-
  DEMO-1003205819
{
  "eventId": "431a30a8-f1c1-4ffe-9441-52da559cc645",
  "eventType": "PRESTAMO_CREADO",
  "eventVersion": 1,
  "source": "eter-prestamos-service",
  "assetKey": "900123456-1:AF-DEMO-1003205819",
  "nitUniv": "900123456-1",
  "placaInventario": "AF-DEMO-1003205819",
  "actor": "79123456",
  "occurredAt": "2026-10-04T01:58:33.592634Z",
  "payload": {
    "tipo": "PRESTAMO_CREADO",
    "idPrestamo": "ae89220b-b599-46bb-b809-15efabea7c3e",
    "nitUniv": "900123456-1",
    "placaInventario": "AF-DEMO-1003205819",
    "numDocumento": "79123456",
    "destinatarioEmail": "79123456@eter.edu.co",
    "assetKey": "900123456-1:AF-DEMO-1003205819",
    "estado": "VIGENTE",
    "fechaSalida": "2026-09-30T20:58:33",
    "fechaDevolucionPactada": "2026-10-02T20:58:33",
    "fechaDevolucionReal": null
  }
}
```

*Figura 2. Evento PRESTAMO_CREADO publicado por el Outbox de préstamos*

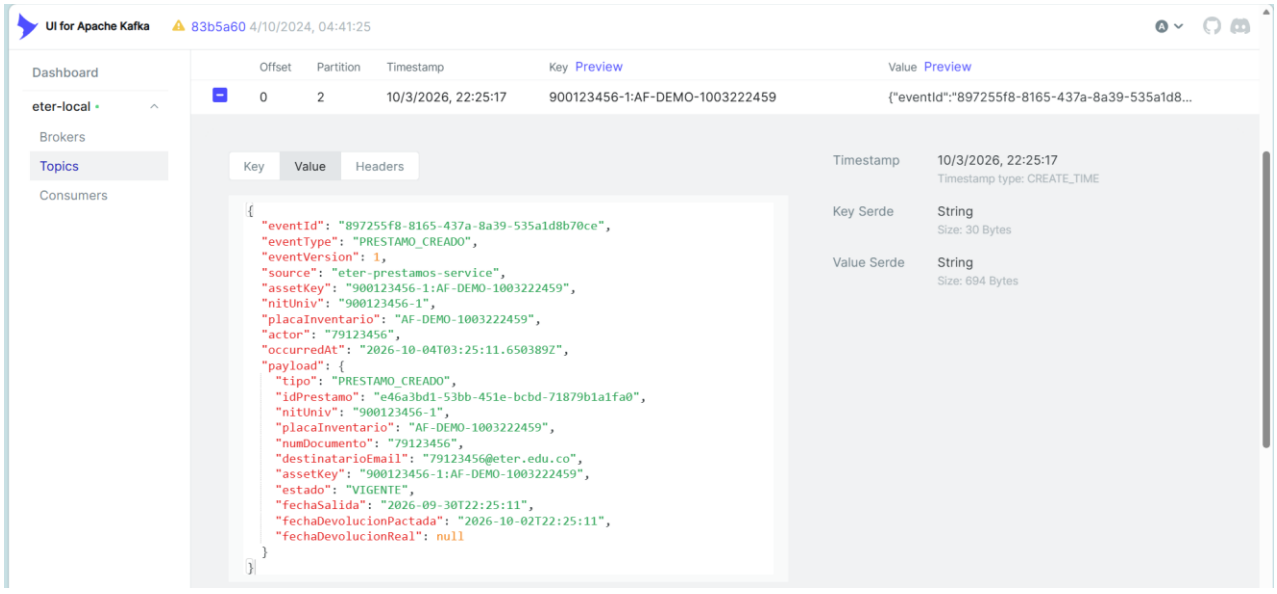



*Figura 3. PRESTAMO_CREADO en Kafka UI con su key, partición y contenido*

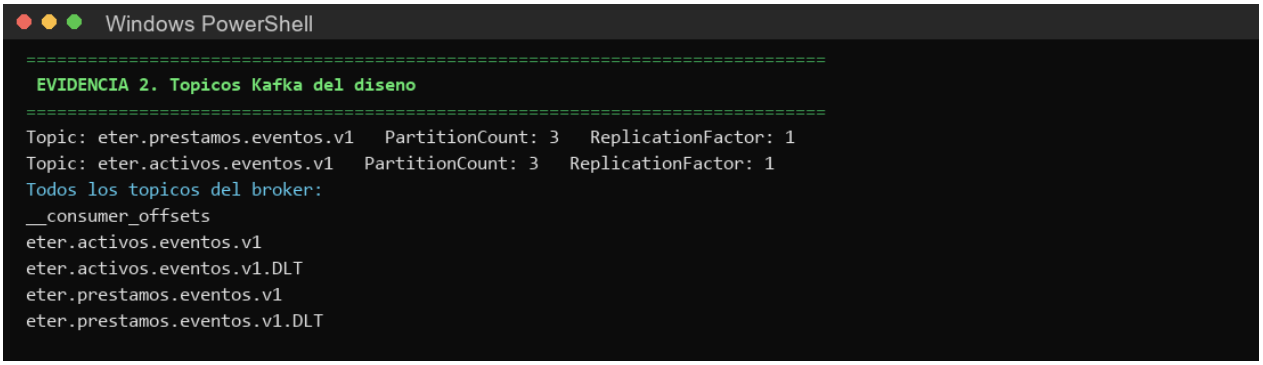



*Figura 4. Tópicos creados por kafka-init*

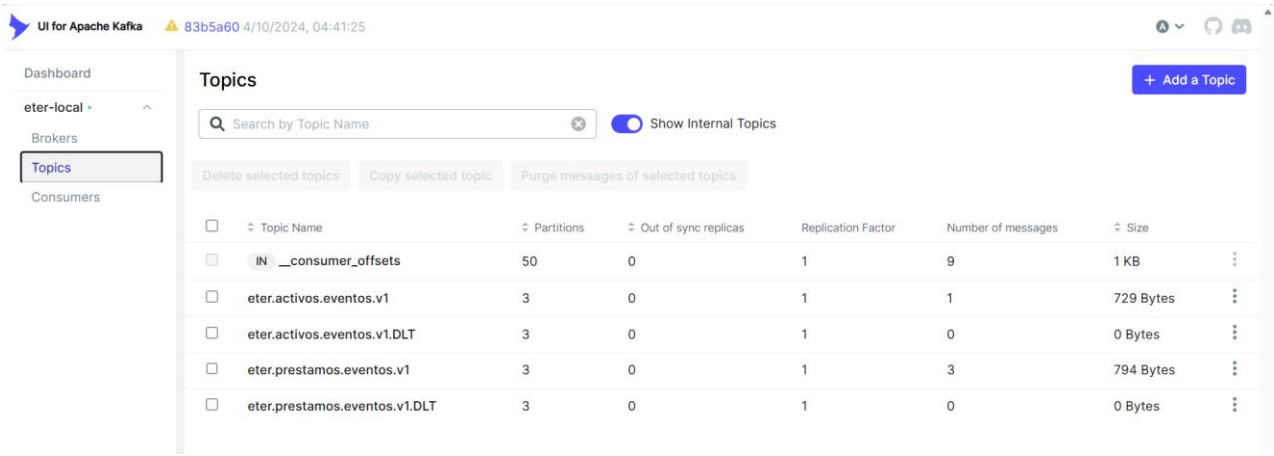



*Figura 5. Tópicos desde Kafka UI*

## 3. Docker Compose: infraestructura y servicios

El archivo infrastructure/docker-compose.yml levanta con un solo comando (**docker compose up -d --build**) PostgreSQL, Kafka (KRaft), el inicializador de tópicos, Redis, Kafka UI y los **cuatro microservicios**, que se compilan dentro de Docker con un Dockerfile multi-etapa (Gradle + JRE 21). Cada servicio tiene healthcheck sobre /actuator/health y espera a que PostgreSQL, Kafka (tópicos creados) y Redis estén listos.

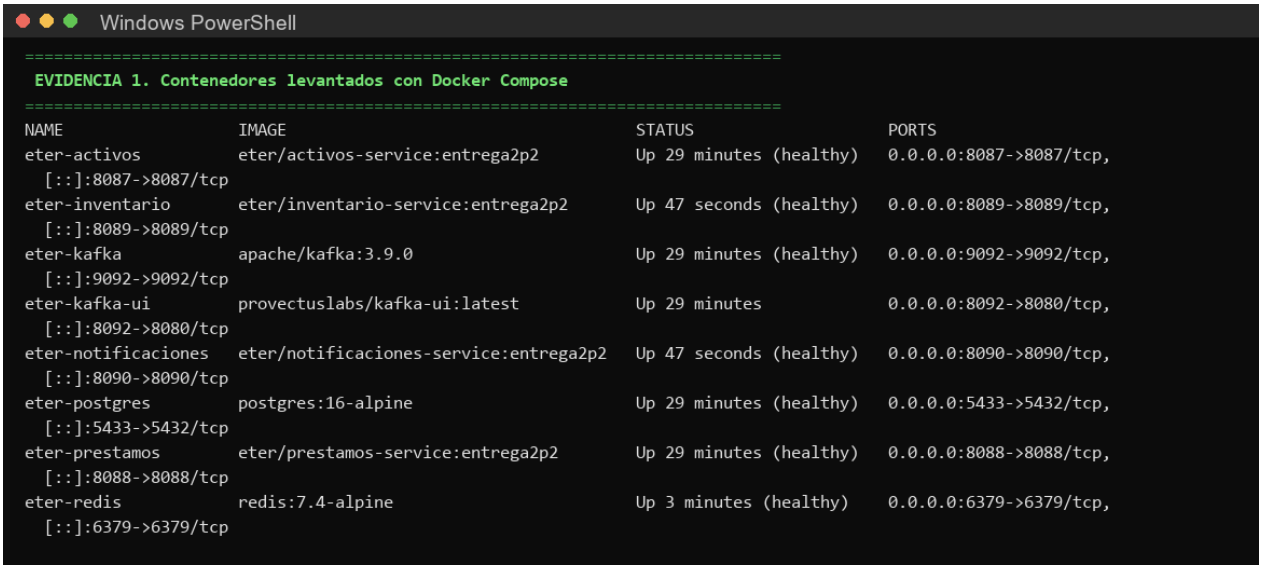



*Figura 6. docker compose ps: los 8 contenedores en ejecución (kafka-init termina tras crear los tópicos)*

## 4. Productores y consumidores funcionales

Las evidencias 3 a 6 y 11 son la salida de **tools/demo_kafka.ps1**, que recorre el flujo completo sobre un activo nuevo. En cada paso se ve el log del productor (evento publicado con su key), el log del consumidor (partición y offset) y el resultado verificable en la API.

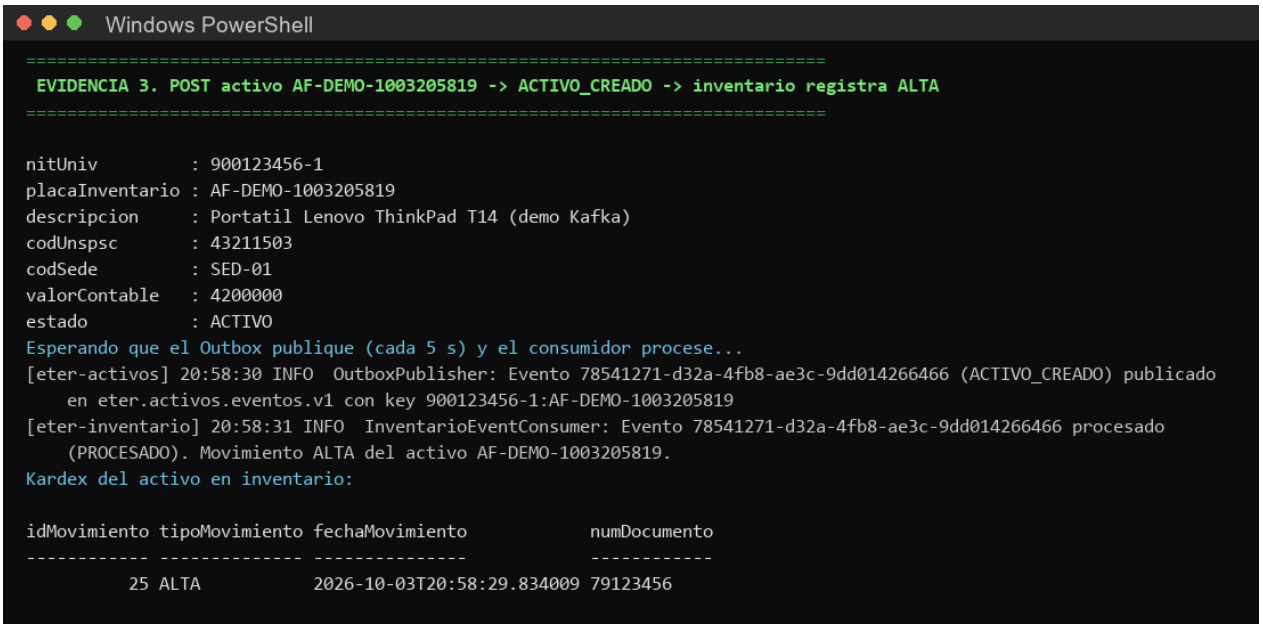



*Figura 7. Alta de activo: ACTIVO_CREADO publicado y consumido por inventario (movimiento ALTA)*

Windows PowerShell

```
================================================================================
 EVIDENCIA 4. POST prestamo (ya vencido) -> PRESTAMO_CREADO -> 2 consumidores
================================================================================

idPrestamo            : ae89220b-b599-46bb-b809-15efabea7c3e
nitUniv               : 900123456-1
placaInventario       : AF-DEMO-1003205819
numDocumento          : 79123456
fechaSalida           : 2026-09-30T20:58:33
fechaDevolucionPactada : 2026-10-02T20:58:33
fechaDevolucionReal   :
estado                : VIGENTE
vencido               : True
[eter-prestamos] 20:58:37 INFO  OutboxPublisher: Evento 431a30a8-f1c1-4ffe-9441-52da559cc645 (PRESTAMO_CREADO)
    publicado en eter.prestamos.eventos.v1 con key 900123456-1:AF-DEMO-1003205819
[eter-inventario] 20:58:37 INFO  InventarioEventConsumer: Evento 431a30a8-f1c1-4ffe-9441-52da559cc645 procesado
    (PROCESADO). Movimiento PRESTAMO del activo AF-DEMO-1003205819.
[eter-notificaciones] 20:58:37 INFO  NotificacionEventConsumer: Evento recibido 431a30a8-f1c1-4ffe-9441-52da559cc645
    [PRESTAMO_CREADO] topico=eter.prestamos.eventos.v1 particion=1 offset=9 key=900123456-1:AF-DEMO-1003205819
Inventario (grupo eter-inventario-v1) - kardex:

idMovimiento tipoMovimiento fechaMovimiento           numDocumento
------------ -------------- ---------------           ------------
          25 ALTA           2026-10-03T20:58:29.834009 79123456
          26 PRESTAMO       2026-09-30T20:58:33        79123456
Notificaciones (grupo eter-notificaciones-v2) - bandeja de 79123456@eter.edu.co:

tipo            titulo                                leida creadaEn
----            ------                                ----- --------
PRESTAMO_CREADO Préstamo registrado: AF-DEMO-1003205819 False 2026-10-04T01:58:38.40916Z
```

*Figura 8. evento PRESTAMO_CREADO lo procesan dos grupos kardex y bandeja*

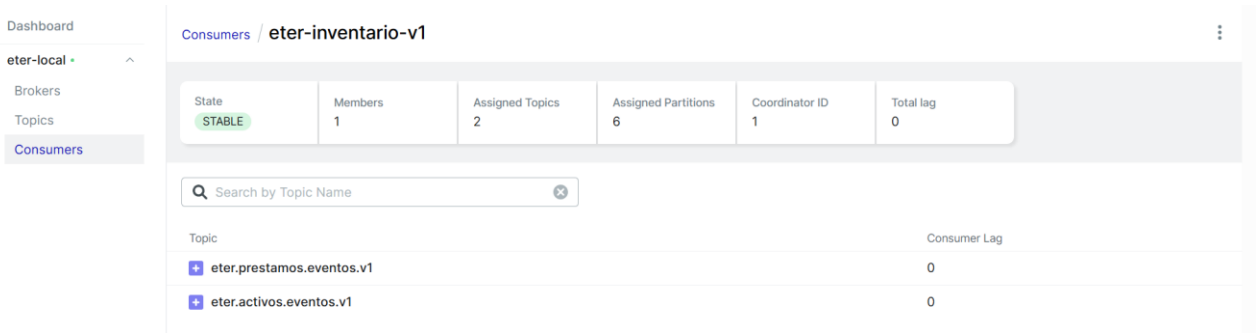



*Figura 9. Grupos de consumidores en Kafka UI*

Windows PowerShell

```
================================================================================
 EVIDENCIA 5. El vigia detecta el vencimiento -> PRESTAMO_VENCIDO
================================================================================
El vigia corre cada 30 s en Docker; esperando...
[eter-prestamos] 20:58:47 INFO  PrestamoVigiaScheduler: Vigía de préstamos: 1 evento(s) PRESTAMO_VENCIDO registrados
    en el Outbox.
[eter-prestamos] 20:58:52 INFO  OutboxPublisher: Evento e2a5b731-356c-434e-bb96-161ead114b91 (PRESTAMO_VENCIDO)
    publicado en eter.prestamos.eventos.v1 con key 900123456-1:AF-DEMO-1003205819
[eter-inventario] 20:58:52 INFO  InventarioEventConsumer: Evento e2a5b731-356c-434e-bb96-161ead114b91
    [PRESTAMO_VENCIDO] no genera movimiento de inventario. Se ignora.

tipo    : PRESTAMO_VENCIDO
titulo  : Préstamo vencido: AF-DEMO-1003205819
mensaje : El activo AF-DEMO-1003205819 no fue devuelto en la fecha pactada (2026-10-02T20:58:33). Por favor regularice
   la devolución.
```

*Figura 10. Vigía emite PRESTAMO_VENCIDO; notificaciones lo procesa e inventario lo ignora*

Windows PowerShell

```
================================================================================
 EVIDENCIA 6. PATCH devolucion -> PRESTAMO_DEVUELTO -> DEVOLUCION + notificacion
================================================================================

idPrestamo          : ae89220b-b599-46bb-b809-15efabea7c3e
placaInventario     : AF-DEMO-1003205819
estado              : DEVUELTO
fechaDevolucionReal : 2026-10-03T20:58:54
Kardex completo del activo (todo lo registro Kafka, nadie lo digito):

idMovimiento tipoMovimiento fechaMovimiento            numDocumento
------------ -------------- ---------------            ------------
          25 ALTA           2026-10-03T20:58:29.834009 79123456
          26 PRESTAMO       2026-09-30T20:58:33        79123456
          27 DEVOLUCION     2026-10-03T20:58:54        79123456
Bandeja del custodio:

tipo              titulo                                leida creadaEn
----              ------                                ----- --------
PRESTAMO_DEVUELTO Préstamo devuelto: AF-DEMO-1003205819   False 2026-10-04T01:58:57.141703Z
PRESTAMO_VENCIDO  Préstamo vencido: AF-DEMO-1003205819    False 2026-10-04T01:58:52.100358Z
PRESTAMO_CREADO   Préstamo registrado: AF-DEMO-1003205819 False 2026-10-04T01:58:38.40916Z
```

*Figura 11. Devolución: PRESTAMO_DEVUELTO genera DEVOLUCION y la notificación; kardex completo*




*Figura 12. Grupos de consumidores al día: LAG = 0 en todas las particiones*

## 5. Procesamiento, integración y persistencia

Los productores usan el patrón **Transactional Outbox**: el cambio de negocio y la fila outbox_event se guardan en la misma transacción, y OutboxPublisher los envía a Kafka con acks=all; solo cuando el broker confirma se marca published_at. Un fallo de Kafka deja el evento pendiente (last_error, publish_attempts) y **no detiene el resto del lote** (error corregido en esta entrega). Los consumidores persisten el resultado en su propio schema.

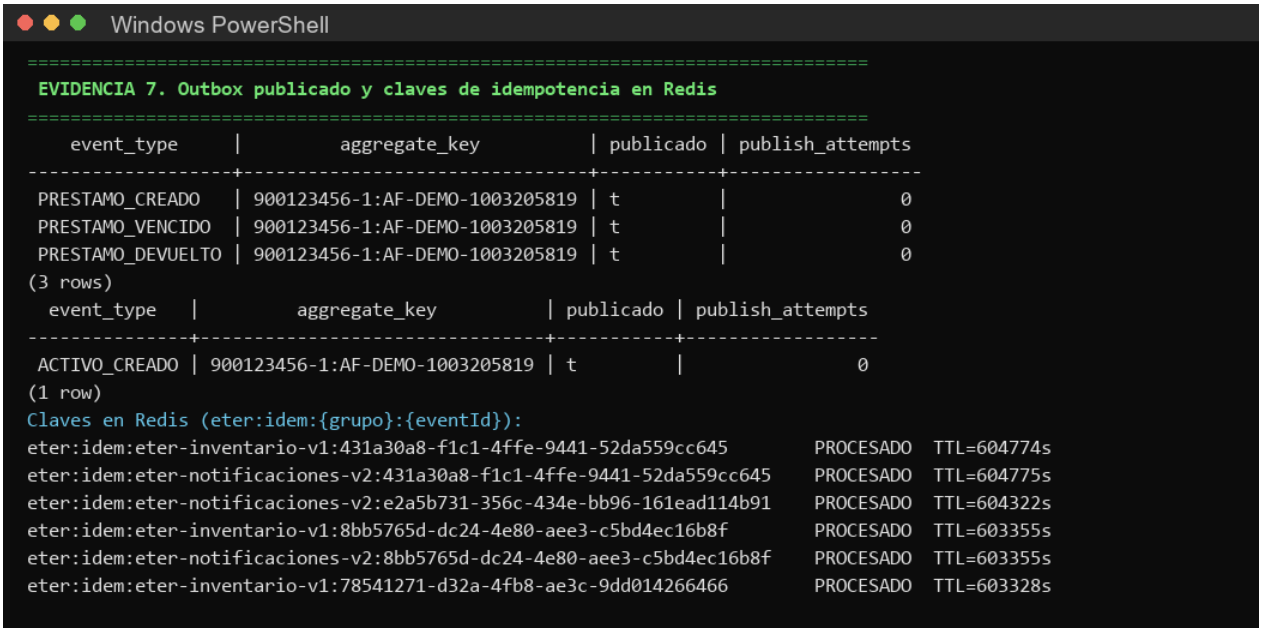



*Figura 10. Outbox de préstamos y activos publicado, y claves de idempotencia en Redis con su TTL*

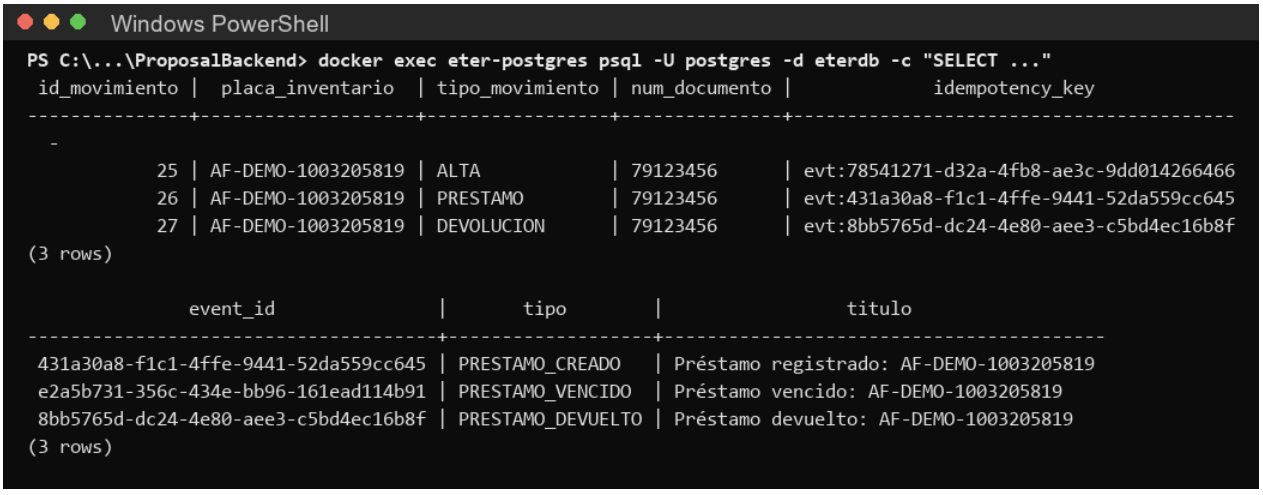



*Figura 11. Filas creadas por los consumidores: movimientos con idempotency_key = evt:{eventId} y notificaciones con event_id*

## 6. Manejo de errores, idempotencia y logs

Kafka garantiza entrega **al menos una vez**, por lo que un mismo evento puede llegar dos veces. Cada consumidor es idempotente en **dos niveles**:

- **Redis (primer filtro):** antes de procesar se ejecuta SET eter:idem:{grupo}:{eventId} PROCESANDO NX EX 300. Si la clave ya está en PROCESADO el evento se descarta sin tocar la base; si está PROCESANDO (otra instancia) se lanza una excepción reintentable; al terminar se marca PROCESADO por 7 días y, si falla, se borra la clave para que el reintento pueda entrar. La clave incluye el grupo, porque cada consumidor debe procesar el evento una vez.
- **PostgreSQL (respaldo):** UNIQUE(event_id, destinatario_email) en notificaciones e índice único (nit_univ, idempotency_key) en inventario. Si Redis no responde, el consumidor sigue (fail-open) y la base frena el duplicado.
- **Errores:** un mensaje inválido (JSON roto, sin eventId, fecha inválida) va **directo** a <tópico>.DLT sin reintentos; un error transitorio se reintenta 6 veces con backoff exponencial (1 s, 2 s, 4 s, 8 s, 10 s, 10 s). Los mensajes del DLT llevan las cabeceras eter-consumer-group y eter-error con la causa.
- **Logs:** cada paso deja un registro (publicado, recibido con partición/offset, procesado, DUPLICADO, WARN Redis no disponible, ERROR enviado al DLT).

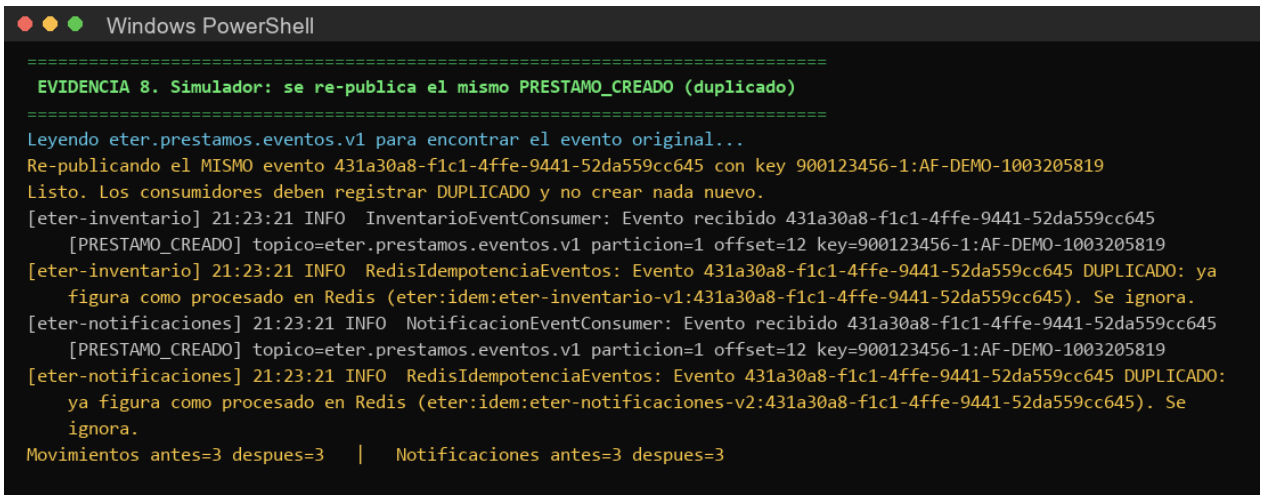



*Figura 12. Simulador: se re-publica el mismo evento; ambos consumidores lo detectan como DUPLICADO en Redis*

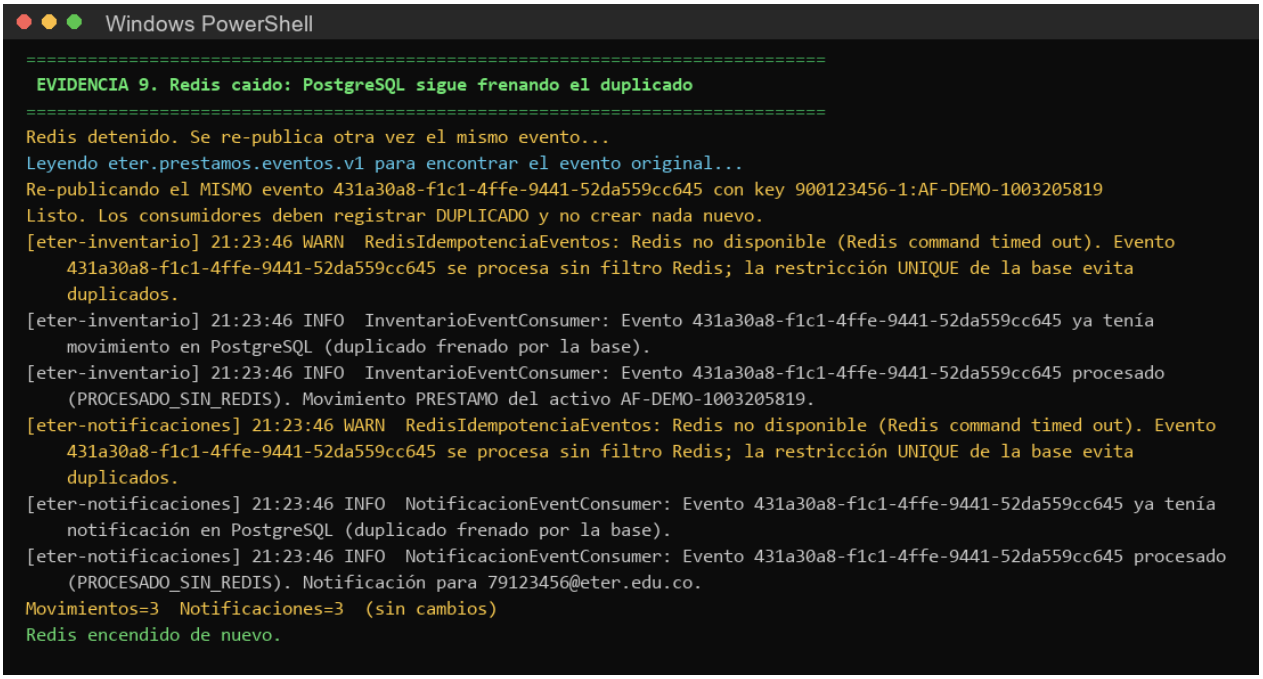



*Figura 13. Redis detenido: WARN en el log y PostgreSQL frena el duplicado; los conteos no cambian*

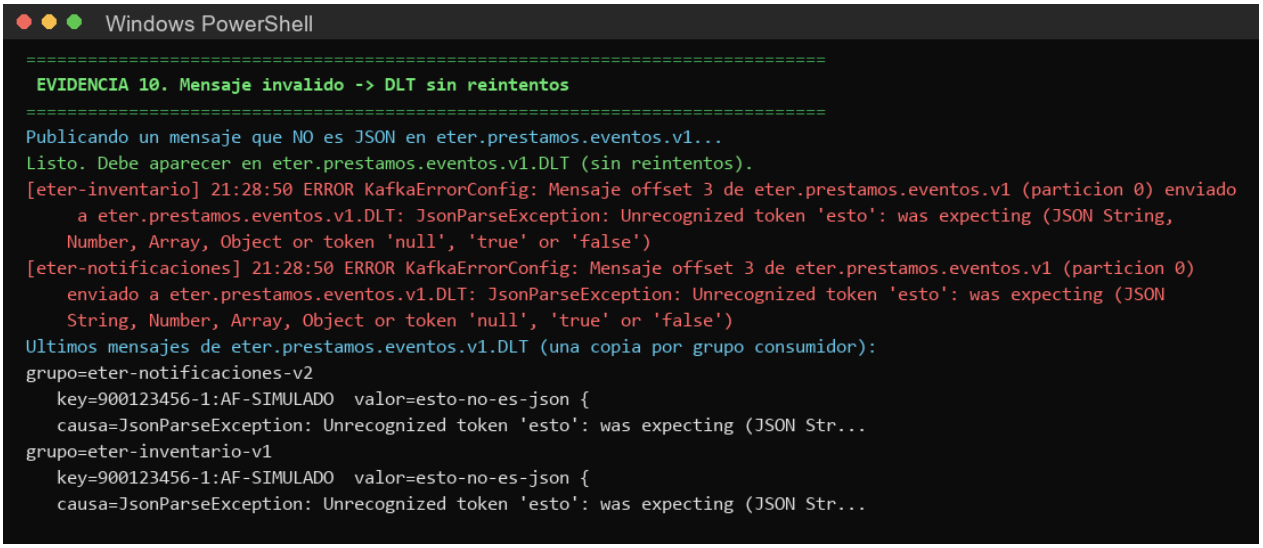



*Figura 14. Mensaje inválido: va directo al DLT con la causa, una copia por grupo consumidor*

## 7. Pruebas

Se agregaron pruebas unitarias para productores, consumidores e idempotencia (56 pruebas en total, todas exitosas) y una carpeta de Postman, **6. Flujo Kafka (Entrega 2P2)**, cuyos tests verifican que cada evento llegó a su consumidor. Se ejecutó con Newman.

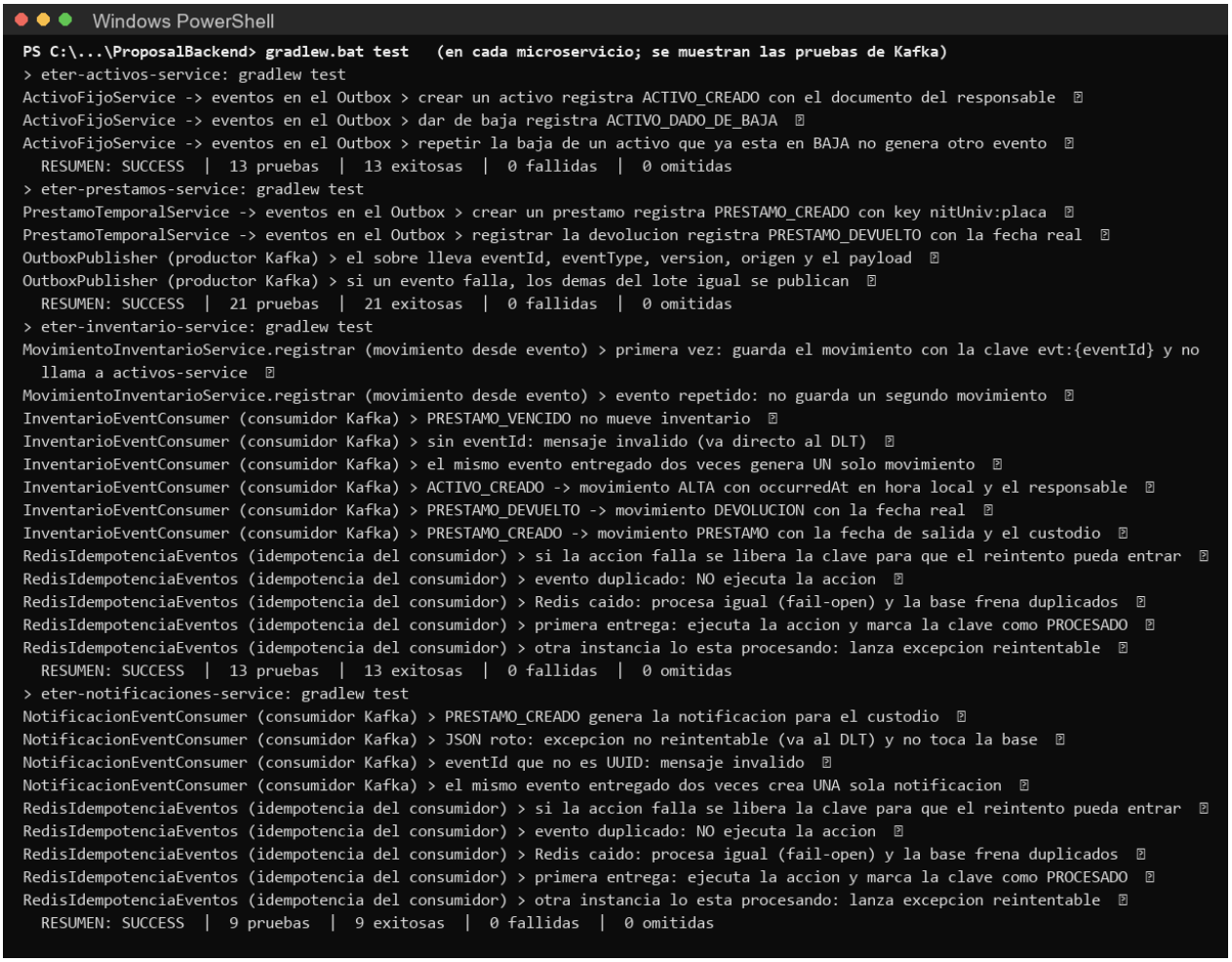



*Figura 15. Pruebas unitarias de Kafka, Outbox e idempotencia: 56/56 exitosas en los 4 servicios*

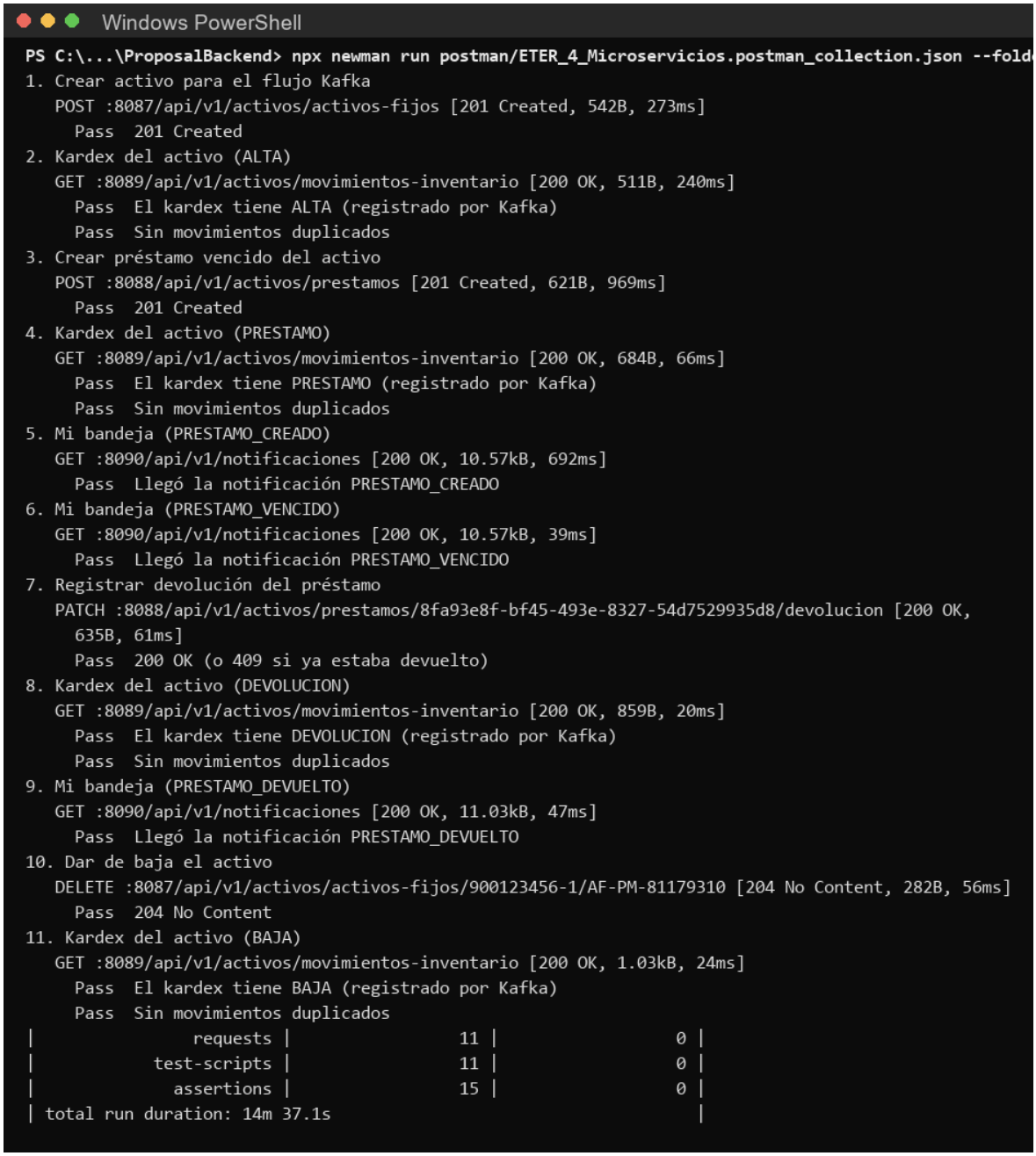



*Figura 16. Colección Postman ejecutada con Newman: 11 requests y 15 aserciones sin fallos*

## 8. Cómo reproducir el flujo

| Paso | Comando (desde ProposalBackend) |
|---|---|
| **1. Levantar todo** | `cd infrastructure && docker compose up -d --build` |
| **2. Demostración completa** | `powershell -ExecutionPolicy Bypass -File tools\demo_kafka.ps1` |
| **3. Simular duplicado** | `tools\simular_evento.ps1 -Tipo duplicado -Buscar PRESTAMO_CREADO` |
| **4. Simular mensaje inválido** | `tools\simular_evento.ps1 -Tipo invalido` |
| **5. Kafka UI** | `http://localhost:8092` |
| **6. Pruebas** | gradlew.bat test` en cada servicio |